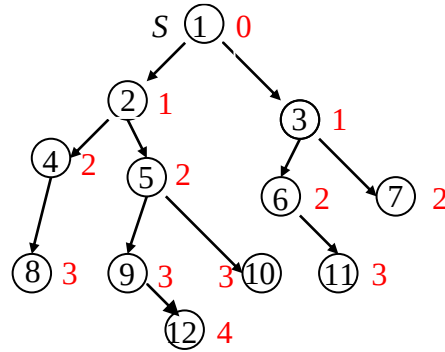
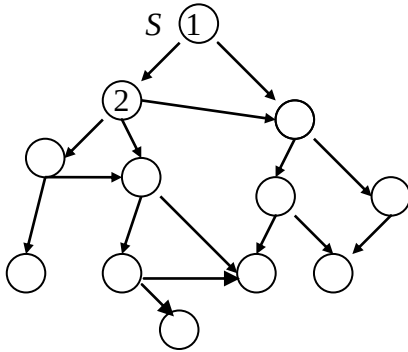


CS 575: Design and Analysis of Algorithms
Solution to Theory Assignment 3.2

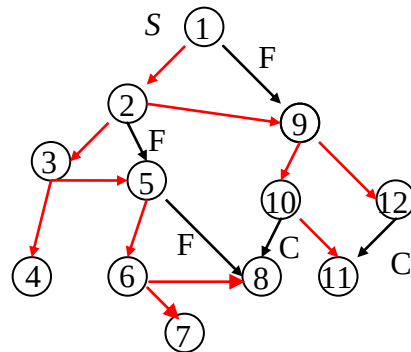
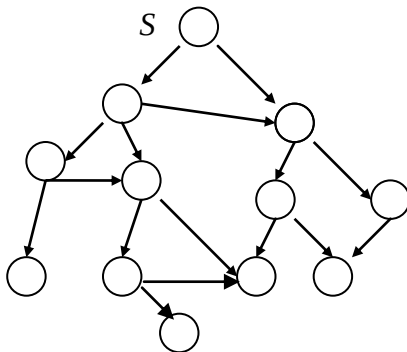
1. [20 points] In this problem, you will apply different traversal methods on a given directed graph. The start node is denoted by S . When arbitrary decisions on order must be made assume that child nodes are visited from left to right.

- a. [10 points] Perform a breadth-first search on the directed graph below (on the left). (i) *Number the nodes* according to the order in which they are visited (become gray). For example, the first node visited is S so S is numbered 1. Then, the left child of S is visited so it is numbered 2. Show the order numbers inside the circles. (ii) Show the distance of each node to S beside each circle. (iii) Show the breadth-first tree.



Answer: The results are shown in the figure on the right above. The numbers in the circles are sequence numbers based on the order of visit. The numbers outside the circles are the distances to S .

- b. [10 points] Perform a depth-first search on the directed graph below (on the left). (i) Number the nodes according to the order in which they are visited. Show the order numbers inside the circles. (ii) Show the depth-first tree(s).



Answer: The results are shown in the figure on the right above. The numbers in the circles are sequence numbers based on the order of visit. Unlabeled (red) edges are the tree edges which form the depth-first tree. The edges marked by F and C are forward edges and cross edges, respectively (I did not ask you to mark this, you can ignore these).

2. [15 points] Please modify the depth first search algorithm (slide 36 and 37) to find all connected components in an undirected graph. Comment on where you made the modification. You need to print out each component ID (starting from 1) and the corresponding vertices. For example, your output for the DFS example (slide 38-41 of the graphs basics lecture notes) will look like the following:
 Component 1: u, v, y, x . Component 2: w, z .

Answer:

DFS (G:graph; **var** color:carray; parent:parray);

```
    for each vertex u do
        color[u]=white; parent[u]=nil;
    end for

    groupID = 1 //Initialize the group ID
    for each vertex u do
        if color[u] == white then
            Print groupID //Print the groupID
            DFS-Visit(u);
            groupID=groupID+1; //The next potential groupID
        end if
    end for
end DFS

DFS-Visit(u)
{
    color[u]=gray;
    print "u" //print out the vertex
    for each v in adj[u] do
        if color[v] = white {
            parent[v] = u;
            DFS-Visit(v);
        }
    color[u] = red;
}
```

3. [25 points] Jack plans to drive from city A to city B along a highway. Suppose $g_0, g_1, g_2, \dots, g_k$ are the gas stations along the highway and are ordered in increasing distance (in miles) from city A , where g_0 is located at the starting place (city A). Let d_i be the distance (in miles) between g_i and g_{i+1} , $i = 0, \dots, k-1$, and we assume that these distances are known to Jack. Each time Jack fills gas to his car, he gets a full tank of gas. Jack also knows the number of miles, m , his car can drive with a full tank of gas. Jack's goal is to minimize the number of times he needs to stop at gas stations for his trip.

- a. [8 points] Design a greedy algorithm to solve the above problem, i.e., to minimize the number of stops for gas.

Answer: A greedy algorithm is to drive as far as possible without running out of gas before the next gas station. More specifically, when Jack starts from a gas station with a full tank, he computes the farthest gas station g_i he can go (i.e., if he attempts to drive to the next gas station g_{i+1} , the car will run out of gas before reaching g_{i+1}).

- b. [12 points] Show that your greedy algorithm has the *greedy choice property*, i.e., each local decision will lead to the optimal solution. Basically you need to argue for the correctness of your algorithm.

Answer: This problem also has the greedy-choice property. Suppose there are r gas stations beyond the start that are within m miles of the start. The greedy solution chooses the r th gas station as its first stop. No station beyond the r th works as a first stop, since Jack would run out of gas first. If a solution chooses a gas station $j < k$ as its first stop, then Jack could choose the r th gas station instead, having at least as much gas when he leaves the r th gas station as if he'd chosen the j th gas station. Therefore, he would get at least as far without filling up again if he had chosen the r th gas station. The same argument can be made after each stop.

- c. [5 points] What is the running time of your algorithm?

Answer: If there are n gas stations, Jack needs to consider each one just once to figure out the gas stations he needs to stop for gas. The running time is $O(n)$.

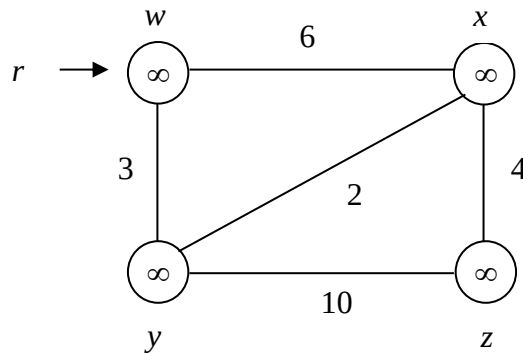
4. [20 points] Given the Prim's algorithm shown below (a min-priority queue is used in the implementation):

```

PRIM( $G, w, r$ )
   $Q = \emptyset$ 
  for each  $u \in G.V$ 
     $u.key = \infty$ 
     $u.\pi = \text{NIL}$ 
  INSERT( $Q, u$ )
  DECREASE-KEY( $Q, r, 0$ )      //  $r.key = 0$ 
  while  $Q \neq \emptyset$ 
     $u = \text{EXTRACT-MIN}(Q)$ 
    for each  $v \in G.Adj[u]$ 
      if  $v \in Q$  and  $w(u, v) < v.key$ 
         $v.\pi = u$ 
        DECREASE-KEY( $Q, v, w(u, v)$ )

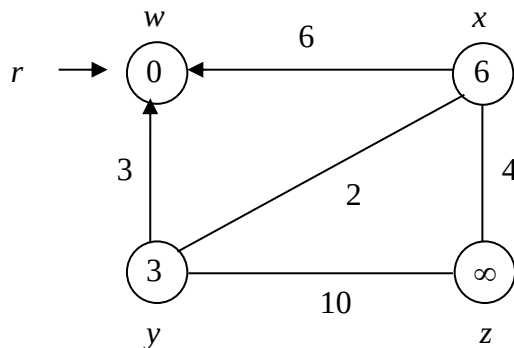
```

Apply the algorithm to the weighted, connected graph below (the initialization part has been done). Show a new intermediate graph after each vertex is processed in the while loop. For each intermediate graph and the final graph, you need to show the vertex being processed, the new key value for each vertex and edges in the current (partial) MST (draw a directed edge from vertex v to u if $v.\pi = u$).

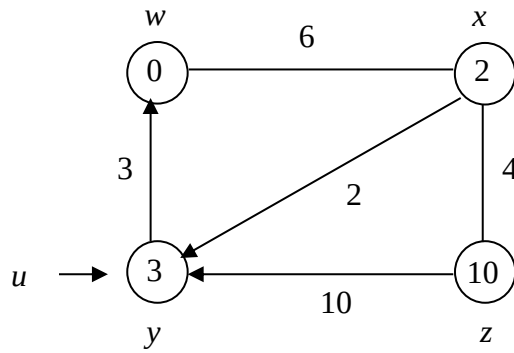


Answer:

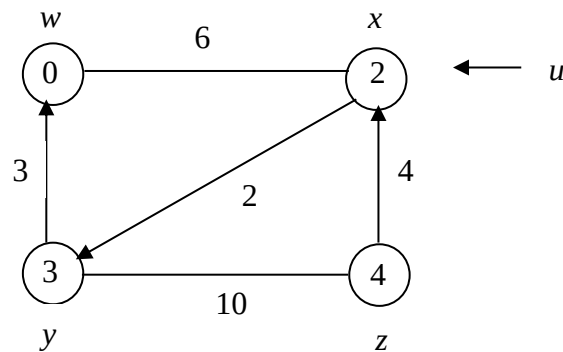
After w is processed:



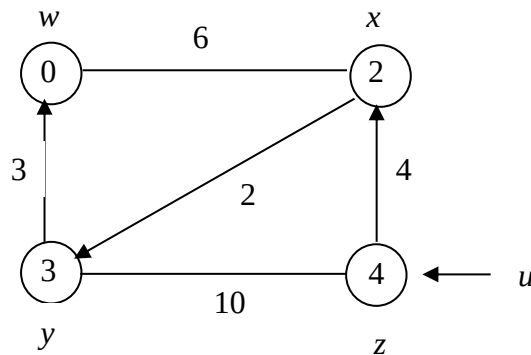
After y is processed:



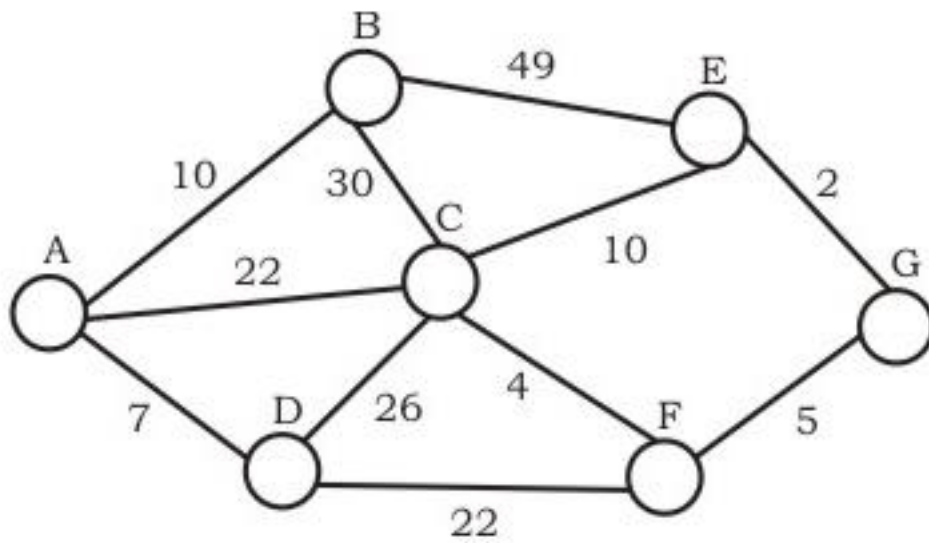
After x is processed:



After z is processed (the final graph):

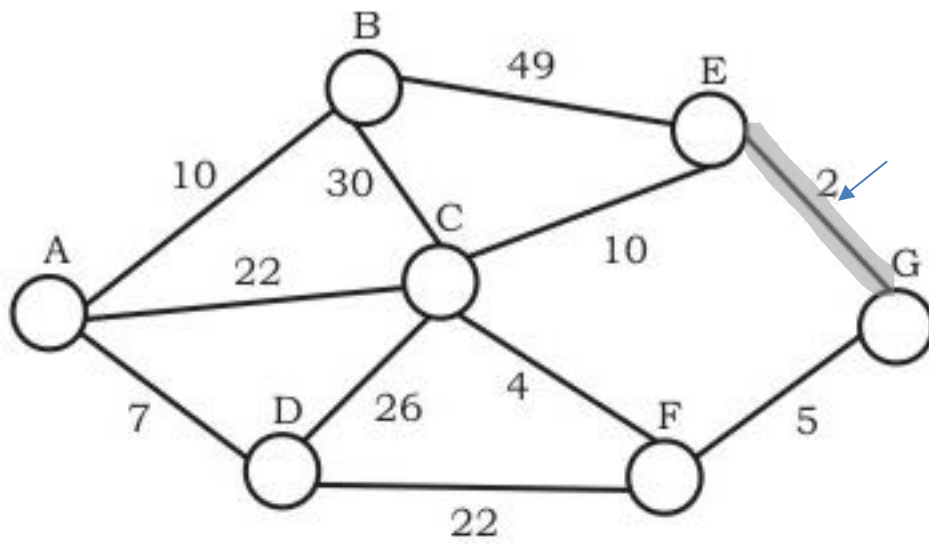


5. [20 points] Apply Kruskal's algorithm to the graph below. Show new intermediate graphs with the shaded edges belong to the forest being grown. The algorithm considers each edge in sorted order by weight. An arrow points to the edge under consideration at each step of the algorithm. If the edges joins two distinct trees in the forest, it is added to the forest, thereby merging the two trees.

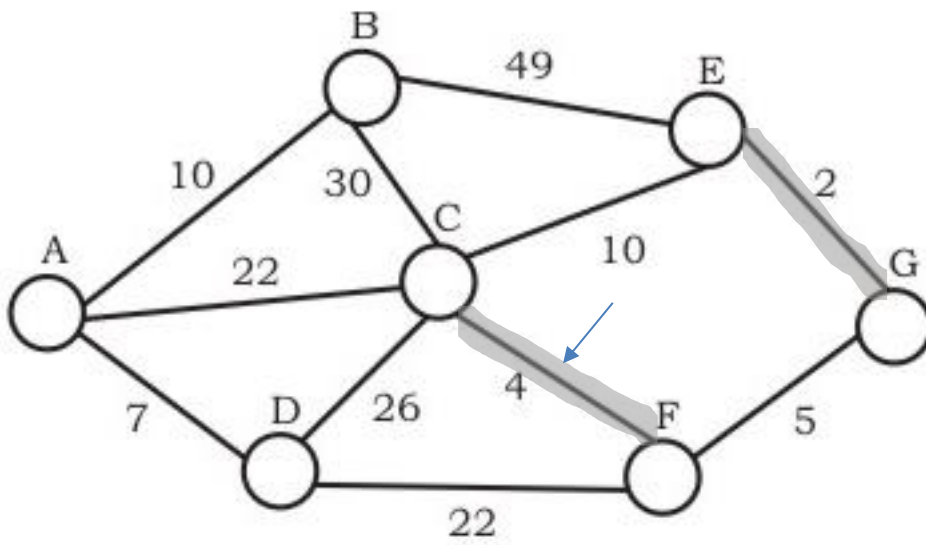


Answer:

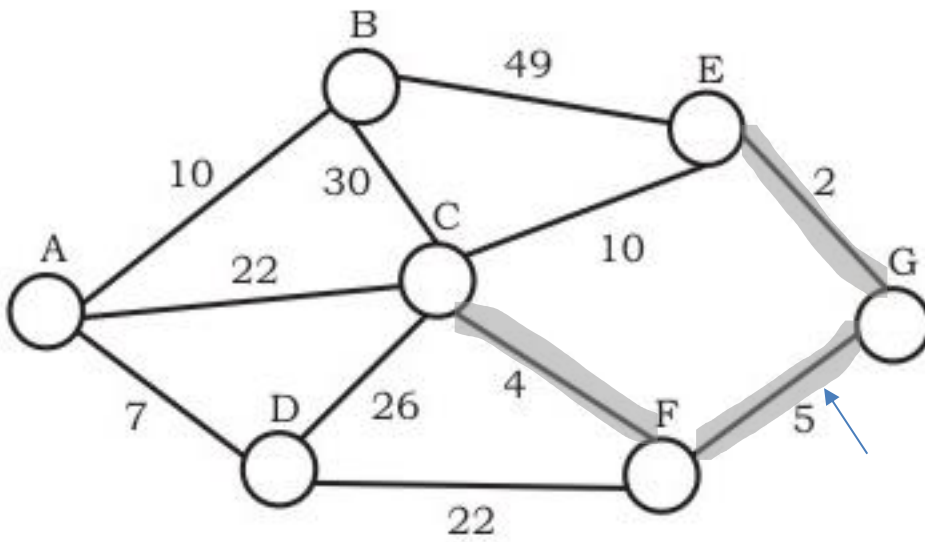
(a)



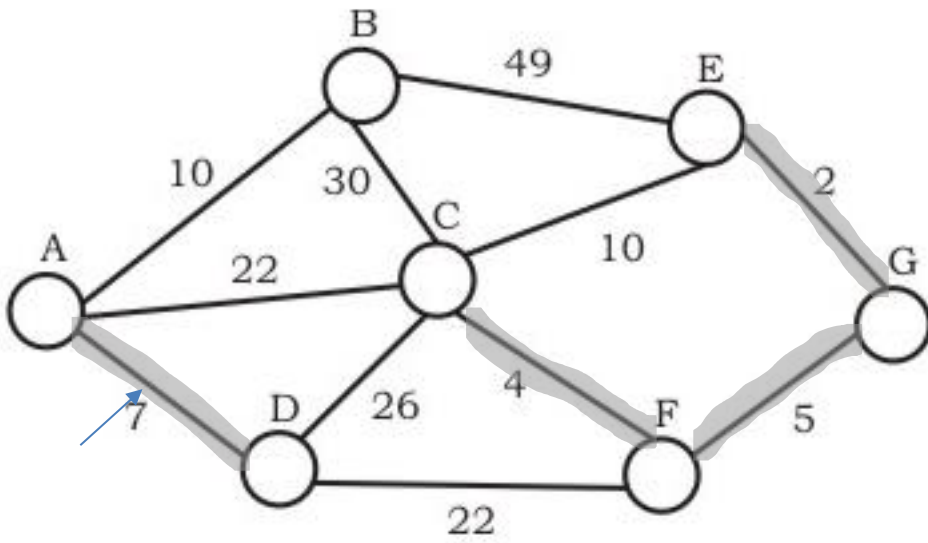
(b)



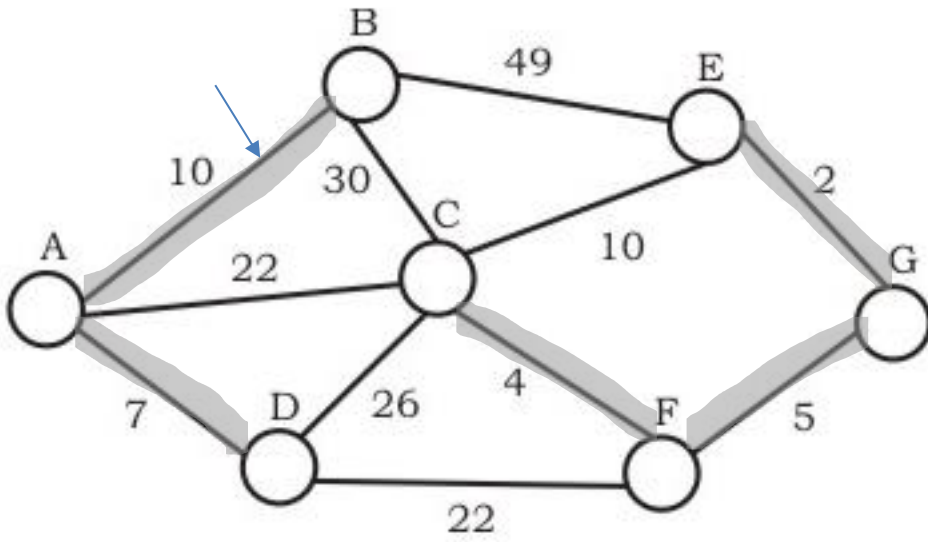
(c)



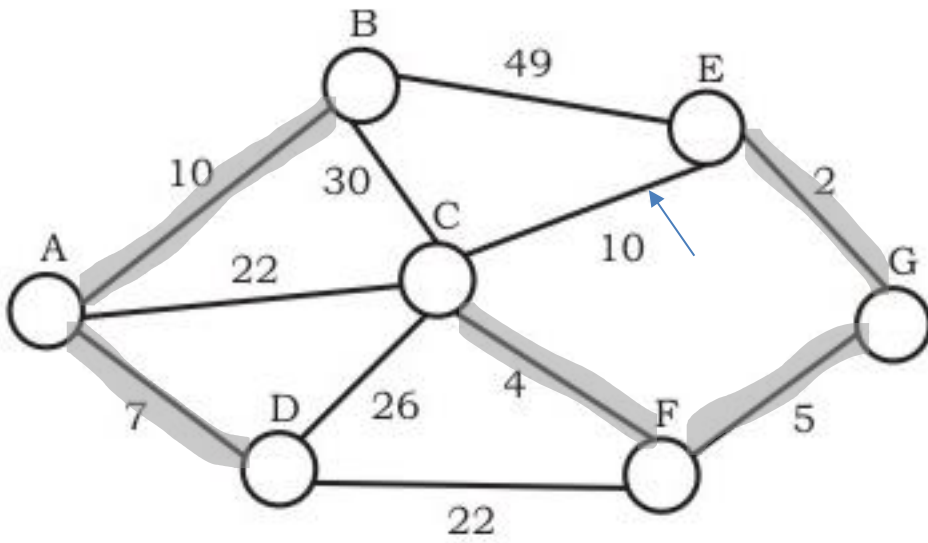
(d)



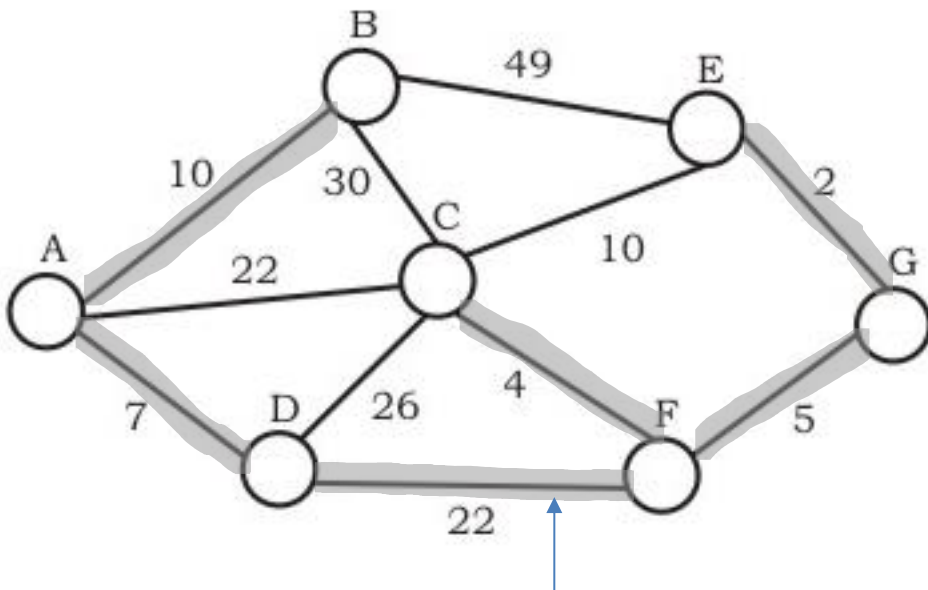
(e)



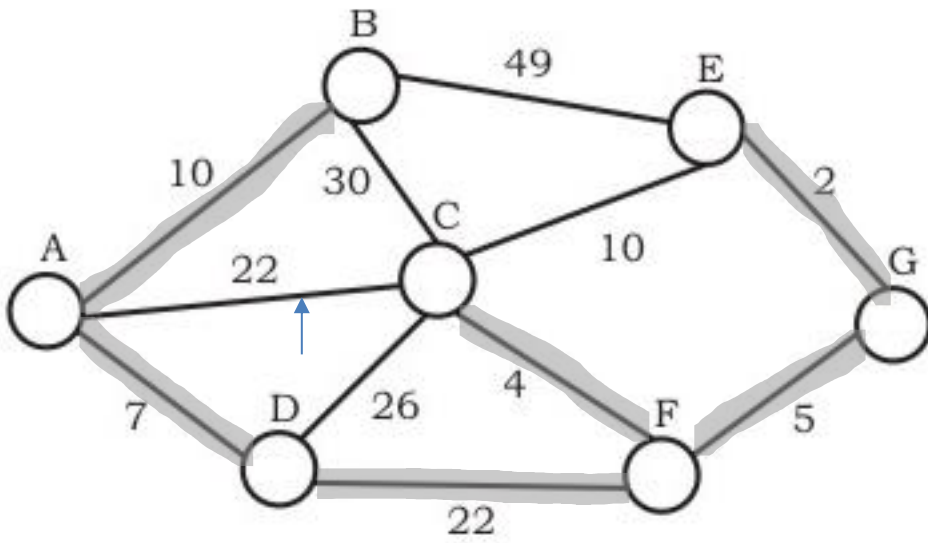
(f)



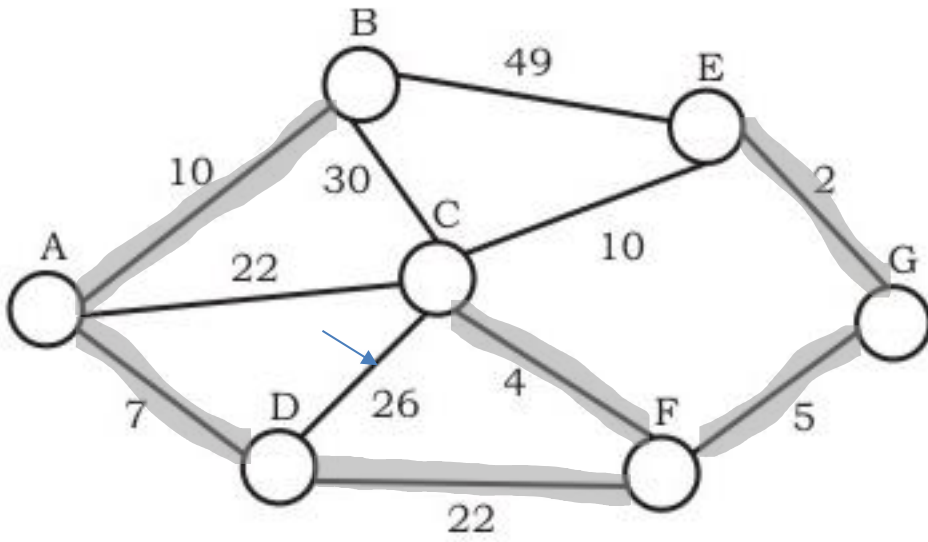
(g)



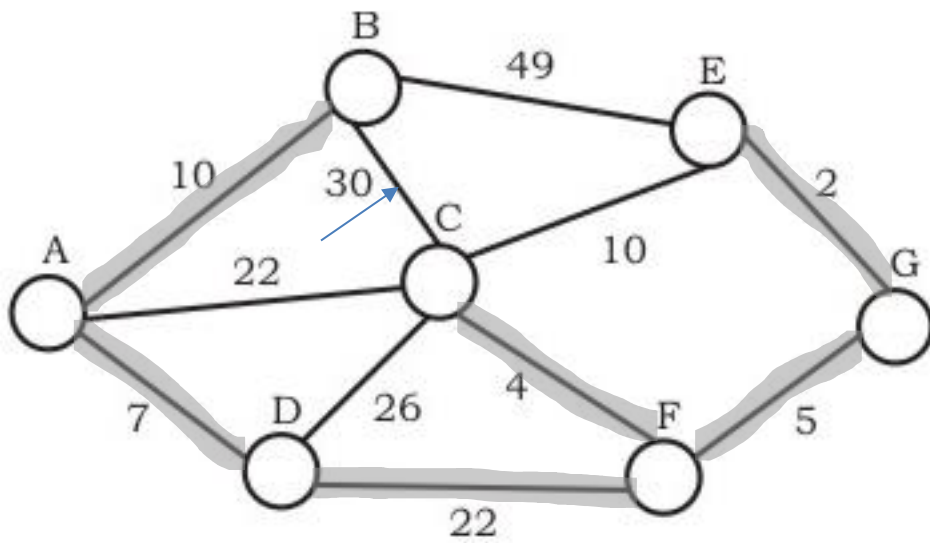
(h)



(i)



(j)



(k)

